

Revisão Final - CI Inovador 2026

Sistemas Digitais + Inglês Instrumental

Base: edital atual, aulas 1-36, prova anterior e erros dos simulados

Objetivo: leitura final antes da prova, com foco no que a banca realmente pode transformar em questão

Prova: 40 questões, 4 horas, presencial

Como usar este arquivo

Este material não substitui as aulas. Ele serve para reativar rapidamente o que você já estudou e impedir que uma palavra, uma fórmula ou uma troca de conceito custe uma questão.

Leitura completa - cerca de 50 a 70 minutos

Leia na ordem. Pare apenas quando não conseguir explicar uma regra com suas próprias palavras.

Leitura rápida - cerca de 25 minutos

Priorize:

1. estratégia de prova;
2. quadro de prioridades pessoais;
3. inglês;
4. temporização, aritmética e CMOS;
5. assembly, pipeline, cache, branch prediction e FPU;
6. folha de fórmulas;
7. checklist final.

Emergência - cerca de 10 minutos

Leia somente:

- "Se eu lembrar apenas de 20 coisas";
- folha de fórmulas e unidades;
- pegadinhas da prova anterior;
- checklist da terceira passada.

1. O perfil da prova

1.1 O enunciado costuma ser maior que o conceito

Na prova anterior, muitas questões apresentavam um cenário longo - banco, indústria, tráfego, áudio, comunicação ou processador - para cobrar uma associação curta:

antecipar dados	→ prefetch
gerar sinais para os blocos	→ unidade de controle
guardar temporários rapidamente	→ banco de registradores
decrementar um valor	→ subtrator
reduzir glitches	→ equilibrar atrasos/remover hazard
selecionar uma entre várias fontes	→ multiplexador
guardar estado e seguir sequência	→ FSM/circuito sequencial

Retire mentalmente a história e pergunte:

Qual mecanismo, bloco ou relação técnica o verbo do enunciado está pedindo?

1.2 O principal risco: alternativa relacionada, mas não direta

Uma alternativa pode ser tecnicamente verdadeira e ainda não responder ao que foi pedido.

Exemplos:

- oscilador fornece ritmo, mas o **subtrator** realiza o decremento;
- cache ajuda o processador, mas a **FPU** é quem opera ponto flutuante;
- reduzir o clock melhora margem de setup, mas não elimina um **hazard de dados**;
- um detalhe aparece no texto em inglês, mas não é o **assunto principal**.

Palavras que exigem atenção:

principal
mais básica
diretamente
fundamental
melhor descreve
incorreta / exceto

1.3 Separe a camada do problema

Camada	Exemplos
Software	algoritmo, compilador, programa
ISA/assembly	instrução, imediato, registrador, branch
Microarquitetura	datapath, controle, cache, pipeline, forwarding

Camada	Exemplos
Circuito digital	portas, MUX, somador, latch, CMOS
Temporização física	atraso, setup, hold, skew, glitch
Sistema de memória	registradores, cache, RAM, memória virtual

Se a pergunta é sobre dependência entre instruções, não escolha uma solução de atraso de portas. Se é sobre carregamento antecipado de dados, não escolha branch prediction.

1.4 Protocolo de seis passos

1. Circule o comando: correta, incorreta, principal, diretamente.
2. Identifique o assunto e a camada.
3. Reduza o enunciado a uma frase curta.
4. Responda mentalmente antes de ler as alternativas.
5. Elimine opções de outra camada ou que acrescentem informação não sustentada.
6. Justifique a escolhida em uma frase.

Se você não consegue formular uma justificativa curta, marque a questão para a segunda passada.

2. Estratégia para as quatro horas

2.1 Três passadas

Primeira passada - 0 a 80 minutos

- Resolva as diretas.
- Marque cada questão como:

✓ = tenho evidência e fecho
 ? = estou entre duas ou falta uma conta
 ! = não identifiquei o caminho

- Não gaste oito minutos tentando salvar uma questão enquanto há pontos fáceis adiante.

Segunda passada - 80 a 150 minutos

- Volte primeiro nas ?.
- Faça tabelas pequenas, contas, rastreios de assembly e eliminação de alternativas.
- Use questões posteriores como pistas de terminologia, sem assumir que a prova sempre é coerente.

Terceira passada - 150 a 195 minutos

Procure somente erros evitáveis:

INCORRETA / EXCETO

- signed versus unsigned
- carry versus overflow
- bit versus byte
- ns versus ps
- endereço versus conteúdo
- estado atual versus próximo estado
- detalhe versus assunto principal
- stall versus flush
- cache versus prefetch versus memória virtual

Folha de respostas e margem - 195 a 240 minutos

- Reserve cerca de 30 minutos para transferência e conferência.
- Deixe aproximadamente 15 minutos de margem.
- Confirme que não pulou uma linha.

2.2 Quando mudar uma resposta

Troque somente se surgir evidência concreta:

- uma fórmula;
- um trecho explícito do texto;
- uma tabela-verdade;
- um rastreio;
- uma incompatibilidade objetiva na alternativa.

“Agora a outra parece mais bonita” não é evidência.

2.3 Regras operacionais do edital

Para não perder pontos por procedimento:

abertura dos portões → 13h30
 fechamento → 14h
 prova → 14h às 18h

Leve:

- documento de identificação original com foto;
- caneta esferográfica de tinta preta.

O edital determina que:

- caderno e folhas-resposta são insubstituíveis;
- a folha definitiva deve ser preenchida com caneta preta;
- rasura, preenchimento incompleto, duas alternativas ou nenhuma alternativa marcada tornam a marcação indevida;
- relógio não pode ser usado; há marcador de tempo na sala;
- lápis, lapiseira, borracha, calculadora, celular e demais eletrônicos devem ficar guardados no envelope fornecido;

- ao final, é obrigatório entregar o caderno de questões e a folha-resposta definitiva.

Classificação:

pontuação final = número de respostas corretas
 mínimo para não reprovar = 20/40
 zerar uma disciplina = eliminação

Como o edital não prevê desconto por erro, não deixe questão em branco. Isso também protege contra zerar um dos blocos.

3. Prioridades pessoais para esta revisão

Seus simulados mostraram uma base ampla já consolidada. O ganho mais provável vem de reduzir trocas conceituais e revisar procedimentos específicos.

Prioridade máxima

main topic versus detalhe no inglês
 conectivos e paráfrase
 setup, hold e sinal do skew
 carry, borrow, overflow e flags
 ripple-carry e unidades de atraso
 saída CMOS versus condição da pull-down
 energia armazenada versus energia retirada da fonte
 CMP, BEQ/BNE e endereço efetivo
 load-use, stall, forwarding e flush
 cache, prefetch, branch prediction e FPU
 D=Q+, PISO, ring, Johnson e one-hot

Manutenção rápida

Você demonstrou bom domínio recente de:

booleana, Karnaugh e síntese
 MUX/decoder/encoder
 Moore versus Mealy
 memórias e lógica programável

Não abandone esses assuntos, mas uma revisão curta basta se você consegue explicar cada regra sem olhar.

4. Inglês instrumental - o bloco que não exige traduzir tudo

4.1 Leia primeiro a pergunta

Descubra o que precisa buscar:

Comando	O que procurar
main topic / main idea	ideia que cobre o texto inteiro
according to the text	evidência explícita
the word X means	sentido contextual
the purpose is	objetivo declarado
refers to	antecedente do pronome
can be inferred	conclusão sustentada, sem invenção

Faça uma leitura rápida do texto e depois procure a evidência específica.

4.2 Assunto principal não é detalhe verdadeiro

Use o teste do título:

Qual alternativa serviria como título para o texto inteiro?

Uma opção que explica apenas uma frase, um exemplo ou uma consequência costuma ser detalhe.

Antes de olhar as alternativas, complete:

O texto é principalmente sobre _____.

Isso reduz a chance de escolher a opção que apenas repete palavras do texto.

4.3 Conectivos por função

Função	Conectivos comuns	Tradução aproximada
Adição	moreover, furthermore, further, in addition, also	além disso
Contraste	however, nevertheless, although, whereas	porém, embora
Causa	because, since, due to	porque, devido a
Consequência	therefore, thus, hence, consequently	portanto, assim
Exemplo	for example, for instance, such as	por exemplo
Resumo	in short, in summary, overall	em resumo

Função	Conectivos comuns	Tradução aproximada
Brevidade/modo	briefly	brevemente, em poucas palavras
Sequência	first, then, finally	primeiro, depois, finalmente

Teste seguro: substitua o conectivo por outro da mesma linha e veja se a relação continua.

Pegadinha real:

further ≈ moreover → adição
in short → resumo

4.4 Vocabulário que gerou dúvida

Palavra	Sentido provável no contexto técnico
avenue / avenues	caminho, via, abordagem, possibilidade
encompass	abranger, incluir
faint / fainter	tênue; mais tênue, menos brilhante/intenso
thus	portanto, assim, desse modo
briefly	brevemente, em poucas palavras; pode introduzir uma síntese, mas não equivale automaticamente a in short
moreover	além disso

Não escolha a tradução isolada mais conhecida: escolha a que preserva a função na frase.

4.5 Paráfrase

Uma alternativa correta pode trocar todas as palavras e manter a ideia. Verifique:

1. quem realiza a ação;
2. qual é a ação;
3. sobre o quê;
4. com qual intensidade ou limitação;
5. se causa, contraste e consequência foram preservados.

Repetir palavras do texto não garante correção. Muitas vezes esse é exatamente o distrator.

4.6 Objetivo, opinião e intensidade

Marcadores de objetivo:

this paper proposes...
the aim of this study is...
we investigate...
this work presents...

Marcadores de opinião ou intensidade:

surprisingly
remarkably
unfortunately
clearly
likely
may / might / could

May não significa certeza. Uma alternativa que transforma possibilidade em fato absoluto costuma estar errada.

4.7 Pronomes e orações relativas

Para **it**, **they**, **this**, **which**, **that** e **who**:

- procure o antecedente compatível mais próximo;
- confira singular/plural;
- confirme se o sentido lógico fecha;
- vírgulas podem indicar informação explicativa, não restritiva.

Se uma questão depender de uma palavra destacada que não aparece no material, ela está incompleta. Não transforme falha de impressão em falta de conhecimento.

4.8 Protocolo do bloco de inglês

1. ler o comando
2. classificar: tema, detalhe, vocabulário, conectivo ou objetivo
3. ler rapidamente o texto
4. formular resposta própria
5. voltar ao trecho exato
6. eliminar opção verdadeira, mas secundária

5. Números, códigos e representação

5.1 Binário, hexadecimal e largura

Pesos binários:

```
... 16 8 4 2 1
```

Cada dígito hexadecimal representa quatro bits:

```
1010 1111 0011 10102 = AF3A16
```

Pesos após a vírgula binária:

```
2-1, 2-2, 2-3... = 1/2, 1/4, 1/8...  
101,0112 = 4+1+1/4+1/8 = 5,375
```

Para converter uma fração decimal para binário, multiplique repetidamente a parte fracionária por 2 e leia, em ordem, as partes inteiras obtidas.

Faixas com n bits:

```
unsigned:      0 até 2n - 1  
complemento 2: -2(n-1) até 2(n-1) - 1
```

O mesmo padrão depende da interpretação:

```
1111 unsigned, 4 bits = 15  
1111 signed, C2      = -1
```

Complemento de 2:

```
inverter os bits + 1  
A - B = A + C2(B)
```

Caso-limite: o menor negativo não possui positivo representável na mesma largura.

5.2 Carry, borrow e overflow

Soma unsigned

Cout final indica que o resultado não coube na largura.

Soma signed em complemento de 2

Há overflow quando:

```
positivo + positivo → resultado negativo
negativo + negativo → resultado positivo
```

Operandos de sinais diferentes não geram overflow em uma soma.

Outra regra:

```
V = carry que entra no bit de sinal XOR carry que sai
```

Subtração

Na convenção ARM e nas aulas:

```
C = 1 → não houve borrow
C = 0 → houve borrow
```

Sempre siga a convenção declarada pela questão, pois outras arquiteturas podem nomear a flag de modo diferente.

5.3 BCD

Cada dígito decimal usa quatro bits:

```
59 decimal → 0101 1001 BCD
```

Padrões `1010` a `1111` são inválidos como um único dígito BCD.

Em soma BCD, se o nibble ultrapassar 9 ou produzir carry, corrige-se com:

```
+0110
```

Não confunda `12` BCD (`0001 0010`) com `12` em binário puro (`1100`).

5.4 Gray

Valores consecutivos mudam apenas um bit.

```
Gray = binário XOR (binário >> 1)
```

Para voltar de Gray, use XOR acumulado a partir do bit mais significativo. Não use pesos posicionais diretamente no código Gray.

5.5 ASCII, paridade e checksum

Valores úteis:

```
'0' = 0x30
'A' = 0x41
'a' = 0x61
diferença maiúscula/minúscula = 32
```

ASCII padrão usa 7 bits; com um bit de paridade, podem ser transmitidos 8 bits.

Valor numérico e caractere não são iguais:

```
7 numérico ≠ caractere '7'
'7' = 0x37 = 55 decimal
```

Um bit de paridade detecta qualquer quantidade ímpar de inversões, mas pode deixar passar uma quantidade par. Ele não corrige o dado.

Uma variante de checksum de 8 bits para soma zero é:

```
S = soma dos bytes mod 256
checksum = (-S) mod 256
```

Se $S=0$, o checksum também é 0. Outros sistemas podem definir checksum de modo diferente; não existe um algoritmo universal com esse nome. Não confunda detecção de erro com garantia absoluta.

6. Portas, booleana e síntese

6.1 Portas indispensáveis

```
AND → 1 apenas se todas forem 1
OR → 1 se pelo menos uma for 1
NOT → inverte
XOR → 1 se as entradas forem diferentes
XNOR → 1 se forem iguais
NAND → AND negada
NOR → OR negada
```

Expressões:

```
XOR = A'B + AB'
XNOR = AB + A'B'
```

Precedência usual:

```
NOT → AND → OR
```

NAND e NOR são universais.

Construções úteis:

```
A' = A NAND A = A NOR A
AB = (A NAND B) NAND (A NAND B)
A+B = (A NOR B) NOR (A NOR B)
A+B com NAND = (A NAND A) NAND (B NAND B)
AB com NOR = (A NOR A) NOR (B NOR B)
```

6.2 Leis que precisam estar automáticas

$$\begin{array}{ll}
 A + 0 = A & A \cdot 1 = A \\
 A + 1 = 1 & A \cdot 0 = 0 \\
 A + A = A & A \cdot A = A \\
 A + A' = 1 & A \cdot A' = 0 \\
 A + A \cdot B = A & A(A+B) = A \\
 A(B+C) = AB+AC & \\
 A+BC = (A+B)(A+C) & \\
 AB+AB' = A & \\
 (A+B)(A+B') = A & \\
 A+A'B = A+B &
 \end{array}$$

Soma booleana não é soma aritmética: $1+1=1$.

6.3 De Morgan e dualidade

$$\begin{array}{l}
 (A+B)' = A'B' \\
 (AB)' = A'+B'
 \end{array}$$

Ao negar um bloco:

1. negue todos os termos;
2. troque AND por OR ou OR por AND.

Dualidade troca $+$ por $\cdot$ e 0 por 1 . O dual de uma expressão não é necessariamente seu complemento.

SOP combina naturalmente com NAND-NAND; POS, com NOR-NOR.

6.4 Mintermos, maxtermos, SOP e POS

$$\begin{array}{l}
 \text{mintermo} \rightarrow \text{linha em que } F=1 \rightarrow \text{produto} \\
 \text{maxtermo} \rightarrow \text{linha em que } F=0 \rightarrow \text{soma}
 \end{array}$$

Para um mintermo:

$$\begin{array}{l}
 \text{bit } 1 \rightarrow \text{variável direta} \\
 \text{bit } 0 \rightarrow \text{variável negada}
 \end{array}$$

Para um maxtermo, a polaridade é inversa:

$$\begin{array}{l}
 \text{bit } 0 \rightarrow \text{variável direta} \\
 \text{bit } 1 \rightarrow \text{variável negada}
 \end{array}$$

Notação:

$$\begin{array}{l}
 F = \Sigma m(\text{índices dos } 1) \\
 F = \Pi M(\text{índices dos } 0)
 \end{array}$$

Z significa alta impedância; não é zero. Em uma tabela de especificação ou Karnaugh, **X** pode significar don't care. Em HDL/simulação, **X** costuma significar valor desconhecido e não pode ser tratado automaticamente como don't care.

Buffer tri-state:

```
habilitado    → Y=D
desabilitado  → Y=Z
```

Se dois drivers do mesmo barramento impõem simultaneamente **0** e **1**, há contenção, não um valor booleano válido.

6.5 Karnaugh

Regras:

```
ordem Gray: 00, 01, 11, 10
SOP: agrupar 1
POS: agrupar 0
grupos: 1, 2, 4, 8, 16...
```

- faça os maiores grupos válidos;
- sobreposição é permitida;
- bordas opostas se tocam;
- os quatro cantos podem formar um grupo;
- diagonal não é adjacência;
- grupo de 3 não existe;
- a variável que muda desaparece;
- um **X** só precisa ser usado se simplificar.
- em SOP, um grupo pode conter **1** e **X**, nunca um **0** especificado;
- em POS, pode conter **0** e **X**, nunca um **1** especificado.

Polaridade do termo extraído:

```
SOP/grupo de 1:
constante 1 → literal direto
constante 0 → literal negado

POS/grupo de 0:
constante 0 → literal direto
constante 1 → literal negado
```

Mapa de índices para **AB** nas linhas e **CD** nas colunas:

		CD			
		00	01	11	10
AB	00	m0	m1	m3	m2
	01	m4	m5	m7	m6
	11	m12	m13	m15	m14
	10	m8	m9	m11	m10

Número de literais restantes em um grupo:

$$n - \log_2(\text{tamanho do grupo})$$

6.6 Fluxo de síntese

```

especificação
→ tabela-verdade
→ mintermos ou maxterms
→ Karnaugh/álgebra
→ expressão mínima
→ implementação pedida

```

Não implemente a forma canônica longa se a questão exige circuito mínimo.

7. Blocos combinacionais

7.1 MUX, DEMUX, decoder e encoder

```

MUX      → várias entradas, uma saída selecionada
DEMUX    → uma entrada, uma saída escolhida entre várias
decoder  → código binário ativa uma linha
encoder   → uma linha ativa gera um código

```

MUX 2:1:

$$Y = S'I_0 + SI_1$$

MUX 4:1:

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

Relações:

```

n seletores → MUX com até 2^n entradas
decoder n→2^n
encoder 2^n→n

```

Se várias entradas do encoder puderem estar ativas ao mesmo tempo, ele precisa de prioridade e, muitas vezes, de um sinal de validade.

Síntese:

decoder $\rightarrow F = \Sigma m(\dots)$ pode ser feita com OR das saídas indicadas
 MUX 4:1 para função de 3 variáveis
 $\rightarrow$ use duas variáveis na seleção
 $\rightarrow$ cada entrada recebe 0, 1, X ou X'

Ativo em nível baixo:

EN_n = 0 $\rightarrow$ enable ativo
 Y_n = 0 $\rightarrow$ saída ativa

7.2 Comparador

Para um bit:

EQ = A XNOR B
 GT = AB'
 LT = A'B

Em palavras maiores, a comparação começa conceitualmente pelos bits mais significativos. A interpretação signed e unsigned pode produzir ordens diferentes.

7.3 Como reconhecer na prova

escolher uma fonte $\rightarrow$ MUX
 ativar uma posição por código $\rightarrow$ decoder
 converter decimal para BCD $\rightarrow$ encoder de 10 entradas e 4 saídas
 distribuir um sinal $\rightarrow$ DEMUX
 comparar magnitude/igualdade $\rightarrow$ comparador

8. FSM, latches, flip-flops e registradores

8.1 FSM

Uma máquina de estados possui:

estado atual + entrada $\rightarrow$ próximo estado
 estado/entrada $\rightarrow$ saída

Moore:

saída = f(estados)

Mealy:

```
saída = f(estado, entrada)
```

Moore costuma ter saídas mais estáveis; Mealy pode reagir no mesmo ciclo à entrada.

Bits mínimos na codificação binária:

```
ceil(log2(N estados))
```

One-hot:

```
N estados → N flip-flops
um único bit 1 por estado válido
```

One-hot gasta mais flip-flops, mas pode simplificar a lógica, especialmente em FPGA.

Para rastrear uma FSM:

1. comece no estado de reset/inicial;
2. consuma uma entrada por vez;
3. siga a transição correspondente;
4. registre a saída no estado, em Moore, ou na transição, em Mealy;
5. verifique se a detecção permite sobreposição antes de descartar bits anteriores.

8.2 Latch SR e D

SR com NOR, ativo alto:

```
S R
0 0 → mantém
1 0 → set
0 1 → reset
1 1 → inválido
```

SR com NAND, entradas ativas em zero:

```
S_n R_n
1 1 → mantém
0 1 → set
1 0 → reset
0 0 → inválido
```

Latch D com enable ativo alto:

```
EN=1 → transparente, Q acompanha D
EN=0 → mantém
```

Com enable ativo baixo:

EN_n=0 → transparente

EN_n=1 → mantém

Latch é sensível a nível; flip-flop, normalmente, à borda.

8.3 Flip-flops

FF D: $Q^+ = D$

FF T: $Q^+ = Q \text{ XOR } T$

FF JK:

00 mantém

01 reset

10 set

11 alterna

Enable em zero mantém o valor mesmo quando chega a borda. Reset síncrono espera clock; reset assíncrono age imediatamente.

8.4 Registradores de deslocamento

SISO → serial entra, serial sai

SIPO → serial entra, paralelo sai

PISO → paralelo entra, serial sai

PIPO → paralelo entra, paralelo sai

Não leia automaticamente a palavra da esquerda para a direita. Se a saída serial é **Q0**, o primeiro bit observado é **Q0**.

8.5 Ring e Johnson

ring:

realimentação direta

n FFs → n estados úteis

Johnson:

realimentação invertida

n FFs → 2n estados úteis

Um ring simples precisa ser inicializado com o padrão correto; em **0000**, pode ficar preso.

9. Contadores e síntese de FSM

9.1 Contadores

n FFs → até 2^n estados

mínimo para contador binário módulo M → $\text{ceil}(\log_2 M)$ FFs

Assíncrono/ripple:

- clock se propaga de um flip-flop ao seguinte;
- acumula atraso;
- é simples.

Síncrono:

- todos recebem o mesmo clock;
- tende a ser mais rápido e previsível.

Em um contador binário completo, divisão de frequência:

```
Q0 = fclock/2
Q1 = fclock/4
Qi = fclock/2^(i+1)
```

Para um contador síncrono binário com FF T:

```
crescente:
T0=1
T1=Q0
T2=Q1.Q0

decrecente:
T0=1
T1=Q0'
T2=Q1'.Q0'
```

Essas relações de divisão e excitação não devem ser aplicadas automaticamente a contador truncado, ring ou Johnson.

Contador módulo 10 usa quatro flip-flops, mas apenas dez dos dezesseis estados possíveis.

9.2 Síntese com FF D

Roteiro:

```
comportamento
→ estados
→ diagrama/tabela
→ codificação
→ equações de próximo estado
→ equações de saída
→ circuito
```

Regra central:

```
D = Q+
```

D é o próximo valor desejado, não obrigatoriamente o valor atual. Se a banca fornece a codificação, use-a. Reset deve levar ao estado inicial e estados não usados precisam de tratamento quando solicitado.

10. Temporização: setup, hold, skew e caminho crítico

10.1 Quatro tempos diferentes

t_{CQ} → atraso da borda do clock até a saída Q
 t_{comb} → atraso da lógica combinacional
 t_{setup} → dado precisa chegar antes da borda de captura
 t_{hold} → dado precisa permanecer depois da borda

Setup verifica o caminho **máximo**: o dado pode chegar tarde demais.

Hold verifica o caminho **mínimo**: o dado pode mudar cedo demais.

10.2 Setup e frequência máxima

Sem skew:

$T_{clock} \geq t_{CQmax} + t_{combmax} + t_{setup}$
 $F_{max} = 1 / T_{clock_min}$

Conversão útil:

$F_{max}(MHz) = 1000 / T_{clock}(ns)$

O caminho combinacional de maior atraso entre registradores é o caminho crítico.

10.3 Hold

Sem skew:

$t_{CQmin} + t_{combmin} \geq t_{hold}$

Diminuir a frequência aumenta o intervalo entre bordas e pode ajudar setup, mas normalmente **não corrige hold**, que ocorre em torno da mesma borda.

Correções típicas de hold:

- adicionar atraso ao caminho mínimo;
- ajustar distribuição de clock/skew;
- usar célula/roteamento apropriado.

10.4 Skew

Se a questão definir:

$skew = chegada\ do\ clock\ no\ destino - chegada\ na\ origem$

então:

Setup:

$$T_{\text{clock}} \geq t_{\text{CQmax}} + t_{\text{combmax}} + t_{\text{setup}} - \text{skew}$$

Hold:

$$t_{\text{CQmin}} + t_{\text{combmin}} \geq t_{\text{hold}} + \text{skew}$$

Resumo mental:

skew positivo → clock chega mais tarde no destino
→ ajuda setup e piora hold

skew negativo → clock chega mais cedo no destino
→ piora setup e ajuda hold

Se a prova usar outra convenção de sinal, siga a definição dada, não a fórmula decorada.

10.5 Metastabilidade

Se a entrada muda perto da borda e viola setup ou hold, o flip-flop pode entrar temporariamente em um estado não resolvido antes de convergir para `0` ou `1`.

violação de setup/hold → risco de metastabilidade

Ela não é uma terceira saída lógica utilizável. Em sinais assíncronos, sincronizadores reduzem a probabilidade de propagação, mas não a tornam matematicamente zero.

10.6 Pegadinhas

- `thold` não entra na fórmula de frequência máxima.
- caminho crítico é o mais lento; caminho de hold perigoso é o mais rápido.
- pipeline pode dividir um caminho longo, mas o estágio mais lento ainda limita o clock.
- atraso de porta e hazard de pipeline pertencem a camadas diferentes.

11. Ponto fixo, ponto flutuante e IEEE 754

11.1 Ponto fixo

Se há `F` bits fracionários:

```
escala = 2^F
valor real = inteiro armazenado / 2^F
inteiro armazenado ≈ valor real × 2^F
resolução = 2^-F
```

Mais bits fracionários:

- melhor resolução;
- menor faixa inteira, se a largura total continuar igual.

Operações na mesma escala:

soma/subtração → escala permanece
multiplicação → produto bruto tem $2F$ bits fracionários

Depois da multiplicação, normalmente é necessário deslocar/arredondar F bits para voltar ao formato original.

Convenções $Q_m.n$ variam quanto à contagem do sinal. Use a definição fornecida.

Para B bits totais e F fracionários:

unsigned:
 0 até $(2^B - 1)/2^F$

signed em C2:
 $-2^{(B-1)/2^F}$ até $(2^{(B-1)} - 1)/2^F$

11.2 IEEE 754 simples

1 bit de sinal
8 bits de expoente
23 bits de fração
bias = 127

Para número normal:

$\text{valor} = (-1)^s \times 1.\text{fração} \times 2^{(E-127)}$

Casos:

$E=0, \text{fração}=0$ → zero
 $E=0, \text{fração} \neq 0$ → subnormal
 $E=255, \text{fração}=0$ → infinito
 $E=255, \text{fração} \neq 0$ → NaN

Conceitos:

overflow → magnitude grande demais
underflow → resultado minúsculo, entrando na região subnormal/zero
inexact → foi necessário arredondar

Ponto flutuante possui grande faixa dinâmica, mas não precisão infinita. Muitos decimais não têm representação binária exata.

11.3 Arredondamento

Bits auxiliares:

G = guard
 R = round
 S = sticky, indica se algum bit descartado posterior era 1

Modo padrão comum:

mais próximo, empate para par

Outros modos IEEE 754:

em direção a zero
 em direção a +infinito
 em direção a -infinito

Não confunda o bit sticky do arredondamento com uma flag persistente de exceção.

12. Somadores, subtratores, comparadores, shifters e ULA

12.1 Half-adder e full-adder

Half-adder:

$S = A \text{ XOR } B$
 $\text{Cout} = A \cdot B$

Full-adder:

$S = A \text{ XOR } B \text{ XOR } \text{Cin}$
 $\text{Cout} = A \cdot B + A \cdot \text{Cin} + B \cdot \text{Cin}$

Com propagate e generate:

$P = A \text{ XOR } B$
 $G = A \cdot B$
 $S = P \text{ XOR } \text{Cin}$
 $\text{Cout} = G + P \cdot \text{Cin}$

12.2 Ripple-carry

Cout de um full-adder → Cin do seguinte

É simples, mas o carry atravessa os estágios. Quando a questão fornece atraso de carry e da soma final:

```
pior soma de n bits =
(n-1) × tcarry + tsoma_final
```

```
Cout final, se cada estágio acrescenta tcarry:
n × tcarry
```

Exemplo:

```
16 bits, tcarry=120 ps, tsoma=180 ps
15×120 + 180 = 1980 ps = 1,98 ns
```

Carry look-ahead reduz a espera usando sinais **P** e **G**.

12.3 Subtrator

Half-subtractor:

```
D = A XOR B
Bout = A'.B
```

Full-subtractor:

```
D = A XOR B XOR Bin
Bout = A'.B + A'.Bin + B.Bin
```

Subtração com somador:

```
A - B = A + B' + 1
Bentrada = B XOR SUB
C0 = SUB
```

Assim:

```
SUB=0 → soma
SUB=1 → subtração
```

Decrementar é subtrair 1. Um oscilador pode fornecer a referência temporal, mas não realiza a operação aritmética.

12.4 Flags

```
Z → resultado igual a zero
N → bit mais significativo do resultado
C → carry final; em subtração ARM, 1 significa sem borrow
V → overflow signed
```

Não tire conclusão signed apenas de **C**, nem conclusão unsigned apenas de **V**.

Na subtração signed:

A-B tem overflow quando A e B têm sinais diferentes
e o resultado tem sinal diferente de A

12.5 Shifts

LSL → desloca à esquerda, entra 0 à direita
LSR → desloca à direita, entra 0 à esquerda
ASR → desloca à direita, replica o sinal
ROR/ROL → bit descartado retorna pelo outro lado

Em ausência de overflow:

<< k → multiplica por 2^k
>> k lógico → divide unsigned por 2^k

ASR é o deslocamento apropriado para preservar o sinal de um número negativo em complemento de 2.

12.6 ULA

A ULA é combinacional:

operandos + controle → resultado + flags

Registradores guardam os operandos/resultados; a ULA executa a operação.

13. Eletricidade básica, MOS e CMOS

13.1 Relações elétricas suficientes para esta prova

$V = R \cdot I$
 $P = V \cdot I$
 $E = P \cdot t$
 $Q = C \cdot V$

Potência é energia por tempo. Energia é o total acumulado.

Não há indicação de que você precise aprofundar Thevenin, Norton, quadripolos, amplificadores operacionais ou transcondutância para a trilha digital.

13.2 MOS como chave

NMOS conduz com gate=1
PMOS conduz com gate=0

pull-down → caminho para GND → saída 0
 pull-up → caminho para VDD → saída 1

Inversor CMOS:

A=0 → PMOS liga, NMOS desliga → Y=1
 A=1 → PMOS desliga, NMOS liga → Y=0

13.3 Redes CMOS

Na pull-down de NMOS:

série → AND das condições de condução
 paralelo → OR das condições de condução

Se a pull-down conduz quando $G=1$, então:

$$Y = G'$$

Esta é uma pegadinha pessoal importante: a expressão da rede pull-down é a condição de saída zero, não a função de saída.

Exemplo:

pull-down conduz para $A(B+C)$
 $Y = (A(B+C))'$
 $Y = A' + B'C'$

Rede pull-up é a dual: troque série por paralelo e vice-versa.

NAND → NMOS em série, PMOS em paralelo
 NOR → NMOS em paralelo, PMOS em série

Contagem em CMOS estático complementar:

NAND/NOR de n entradas → $2n$ transistores
 AND/OR → NAND/NOR correspondente + inversor

Se nenhuma rede conduz, a saída fica flutuante. Se pull-up e pull-down conduzem ao mesmo tempo, surge um caminho direto entre VDD e GND.

13.4 Potência dinâmica, estática e energia

$P_{din} \approx \alpha \cdot C \cdot VDD^2 \cdot f$
 $P_{estática} \approx VDD \cdot I_{leak}$

Consequências:

C dobra → P_{din} dobra
 f dobra → P_{din} dobra
 α dobra → P_{din} dobra
 VDD dobra → P_{din} quadruplica

Leakage é corrente de fuga em transistores reais e contribui para potência estática. Durante a transição também pode haver breve corrente de curto-circuito porque PMOS e NMOS conduzem simultaneamente.

Energia de um capacitor:

armazenada: $E_{cap} = 1/2 \cdot C \cdot V^2$
 retirada da fonte no carregamento 0→1: $E_{fonte} = C \cdot V^2$

A fonte fornece o dobro do que fica armazenado; a outra metade é dissipada durante o carregamento.

13.5 Atraso, fanout e layout

- atrasos em série se somam;
- fanout maior aumenta a capacitância;
- mais capacitância tende a aumentar atraso e potência;
- interconexões longas aumentam atraso;
- o maior atraso define o caminho crítico.

A prova anterior relacionou redução de atraso à otimização de layout e caminhos, não à memória virtual ou a uma função de software.

13.6 Hazards elétricos e glitches

hazard estático-1 → deveria ficar 1, cai brevemente para 0
 hazard estático-0 → deveria ficar 0, sobe brevemente para 1
 hazard dinâmico → deveria mudar uma vez, oscila várias vezes

Glitches surgem principalmente quando caminhos reconvergentes possuem atrasos diferentes.

Termo de consenso:

$AB + A'C \rightarrow$ adicionar BC
 $(A+B)(A'+C) \rightarrow$ adicionar (B+C)

O termo pode ser logicamente redundante, mas fisicamente útil para remover o hazard.

Não confunda:

hazard elétrico → atraso desigual e glitch
 hazard pipeline → conflito/dependência entre instruções

14. Memórias e lógica programável

14.1 Capacidade de memória

n bits de endereço → 2^n posições
 capacidade em bits = posições × bits por palavra
 8 bits = 1 byte

Para seleccionar **N** registradores:

$\text{ceil}(\log_2 N)$ bits de selecção

14.2 Hierarquia

registradores → muito rápidos, poucos, caros por bit
 cache, tipicamente em SRAM → rápida, pequena, sem refresh
 DRAM → mais densa, volátil, RAM principal, precisa de refresh
 armazenamento → maior e mais lento, não volátil

SRAM é chamada “estática” porque não precisa de refresh, mas continua sendo volátil.

ROM é não volátil e pode implementar uma tabela-verdade:

entradas → endereço
 saídas → palavra armazenada

14.3 PROM, PLA e PAL

PROM → AND/decoder fixo, OR programável
 PLA → AND programável, OR programável
 PAL → AND programável, OR fixo

Família ROM:

PROM → normalmente programada uma vez
 EPROM → apagada por luz ultravioleta
 EEPROM → apagada e regravada eletricamente
 Flash → apagada/regravada em blocos

PLA é mais flexível. PAL tende a ser mais simples, com menos flexibilidade.

14.4 CPLD, FPGA e ASIC

CPLD → controle/glue logic, menor capacidade, atraso previsível
 FPGA → LUTs, FFs, interconexões, RAM/DSP, reconfigurável
 ASIC → fixo, alto custo inicial, ótimo desempenho/energia em volume

LUT de k entradas:

2^k posições
implementa qualquer função de até k variáveis

FPGA baseado em SRAM perde a configuração ao desligar e precisa carregar um bitstream ao ligar.

HDL descreve hardware. FPGA configura uma estrutura de hardware; ele não é simplesmente um processador executando linhas de código, embora possa conter um processador.

15. Cache, prefetch e memória virtual

15.1 Cache

Cache é uma memória pequena e rápida entre o processador e níveis mais lentos.

```
hit      → bloco encontrado
miss     → bloco ausente
hit rate → hits / acessos
miss rate = 1 - hit rate
```

Ela funciona por localidade:

```
temporal → um dado usado tende a ser usado novamente
espacial → endereços próximos tendem a ser usados
```

Um bloco/linha contém vários bytes contíguos.

15.2 Mapeamento

```
direto:
cada bloco tem uma única linha possível

totalmente associativo:
bloco pode ocupar qualquer linha

associativo por conjunto:
bloco escolhe um conjunto e pode ocupar qualquer via dele
```

Compromisso:

- direto é simples e rápido, mas sofre mais misses de conflito;
- totalmente associativo reduz conflitos, mas compara muitas tags;
- por conjunto fica entre os dois.

Campos do endereço:

```
offset = log2(bytes por bloco)
número de conjuntos = capacidade / (bloco × vias)
índice = log2(número de conjuntos)
tag = bits restantes
```

Políticas de escrita:

```
write-through → atualiza próximo nível a cada escrita
write-back    → marca linha dirty e escreve ao substituí-la
```

Políticas de substituição podem incluir LRU, pseudo-LRU, FIFO ou aleatória.

Pegadinhas:

- hit rate não é o tempo de acesso;
- associativa por conjunto não é totalmente associativa;
- cache costuma usar SRAM; RAM principal, DRAM;
- aumentar cache não garante melhoria universal.

15.3 Prefetch

Prefetch tenta trazer dados ou instruções **antes** da demanda.

```
next-line → busca o próximo bloco
stream    → acompanha uma sequência
stride    → reconhece diferença constante entre endereços
```

Métricas:

```
accuracy   = prefetches usados / prefetches feitos
coverage   = misses evitados / misses originais
timeliness = chegou cedo o bastante, mas não cedo demais
```

Distância é quão à frente buscar; grau é quantos blocos buscar.

Prefetch pode atrapalhar:

- polui a cache;
- consome largura de banda;
- gasta energia;
- pode chegar tarde;
- pode expulsar dado útil.

Portanto, “prefetch sempre melhora desempenho” é falso.

15.4 Memória virtual

Cada processo usa endereços virtuais. A MMU os traduz para endereços físicos por meio de tabelas de páginas; a TLB guarda traduções recentes.

```
endereço virtual
→ consulta TLB/tabela de páginas
→ endereço físico
→ cache/RAM
```

Se a página necessária não estiver na RAM, ocorre page fault e o sistema operacional pode buscá-la no armazenamento.

Analogia válida com a ideia de container: cada processo vê seu próprio espaço de endereços e ganha isolamento. Mas memória virtual não é um container completo; é um mecanismo específico de endereçamento, proteção e gerenciamento de memória.

Não confunda:

registrador	→ temporário dentro da CPU
cache	→ cópia rápida de blocos usados
prefetch	→ busca antecipada
memória virtual	→ tradução e isolamento de endereços

Na prova anterior, memória virtual apareceu várias vezes como alternativa de outra camada.

16. Assembly, formatos e endereçamento

16.1 Notação mínima

```
instrução = opcode + operandos
R1         = conteúdo do registrador R1
[R1]       = memória no endereço contido em R1
#5         = imediato de valor 5
```

```
LOAD/LDR → memória para registrador
STORE/STR → registrador para memória
```

Exemplo:

```
ADD R1,R2,R3 → R1 = R2 + R3
LDR R1,[R2]  → R1 = memória[R2]
STR R1,[R2]  → memória[R2] = R1
```

STR não altera o valor de **R1**; ele usa esse valor como dado a escrever.

16.2 Formato de instrução

Campos possíveis:

```
opcode, registradores, imediato, deslocamento, função
```

Dimensionamento de campos:

```
N registradores → ceil(log2 N) bits por campo de registrador
M opcodes       → ceil(log2 M) bits para identificar a operação
bits restantes  → podem formar imediato, função ou deslocamento
```

Tipos conceituais:

```
R → operandos em registradores
I → imediato ou offset
J/jump → salto incondicional, conforme o formato da ISA
branch condicional → normalmente usa deslocamento PC-relative;
                    o formato exato depende da ISA
```

Imediato é a constante codificada na própria instrução, não um dado buscado na memória.

16.3 Modos de endereçamento

imediato	→ valor está na instrução
registrador	→ valor está no registrador
direto	→ instrução contém o endereço
indireto	→ registrador contém o endereço
base + offset	→ EA = base + deslocamento
índice escalado	→ EA = base + índice × tamanho
PC-relative	→ alvo = PC + deslocamento

Palavras de 32 bits ocupam quatro bytes:

$$\text{índice} \ll 2 = \text{índice} \times 4$$

Exemplo:

base 0x1000, índice 5, word de 4 bytes
 EA = 0x1000 + 5×4 = 0x1014

Offset costuma estar em bytes, a menos que a questão diga “palavras”, “halfwords” ou outra escala.

16.4 Branch com imediato escalado

Imediato signed de k bits:

$$-2^{(k-1)} \text{ até } 2^{(k-1)}-1$$

Se representa palavras de quatro bytes, multiplique os extremos por 4.

Exemplo com 12 bits:

-2048 a +2047 palavras
 -8192 a +8188 bytes

O máximo positivo não é simétrico ao negativo.

16.5 Endianness e alinhamento

Endianness define a ordem dos **bytes** de uma palavra na memória:

little-endian → byte menos significativo no menor endereço
 big-endian → byte mais significativo no menor endereço

Não muda a ordem dos bits dentro de cada byte.

Alinhamento significa colocar um dado em endereço adequado ao seu tamanho. Acesso desalinhado pode ser proibido, mais lento ou tratado pelo hardware, conforme a arquitetura.

17. Fluxo, flags, funções e assembly prático

17.1 CMP e branches

`CMP A,B` → calcula $A-B$ para atualizar flags

Não armazena o resultado em `A`.

Branches:

`BEQ` → $Z=1$
`BNE` → $Z=0$

Signed:

`BLT` → $N \neq V$
`BGE` → $N = V$
`BGT` → $Z=0$ e $N=V$
`BLE` → $Z=1$ ou $N \neq V$

Unsigned:

`BLO/BCC` → $C=0$
`BHS/BCS` → $C=1$
`BHI` → $C=1$ e $Z=0$
`BLS` → $C=0$ ou $Z=1$

`BNE` não compara sozinho; ele consulta as flags produzidas anteriormente.

17.2 Laços

Exemplo:

```
SUBS R1,R1,#1
BNE loop
```

`SUBS` decrementa e atualiza flags. `BNE` volta enquanto $Z=0$, isto é, enquanto o resultado ainda não é zero.

Ao rastrear:

1. anote registradores iniciais;
2. execute uma linha por vez;
3. atualize flags apenas quando a instrução fizer isso;
4. decida o branch;
5. controle o número de iterações.

17.3 Chamadas e pilha

```
BL alvo → desvia e salva retorno em LR
BX LR   → retorna
SP      → topo da pilha
PUSH    → salva na pilha
POP     → restaura
```

Uma função que faz outro `BL` pode sobrescrever `LR`; por isso funções não folha normalmente preservam o endereço de retorno.

```
caller-saved → chamador preserva se precisar
callee-saved → função chamada deve restaurar
```

`PUSH` e `POP` precisam permanecer equilibrados.

17.4 Otimização e depuração

Otimização segura preserva o comportamento. Pode envolver:

- manter temporários em registradores;
- reduzir loads/stores redundantes;
- retirar trabalho invariável do laço;
- atualizar ponteiro em vez de recalcular endereço;
- desenrolar laços com cuidado;
- melhorar localidade;
- reduzir dependências e branches.

```
contagem estática → instruções presentes no código
contagem dinâmica → instruções realmente executadas
```

Um laço curto pode ter pequena contagem estática e enorme contagem dinâmica.

Mais instruções não significam automaticamente pior desempenho; menos instruções também não garantem programa mais rápido. Considere memória, cache, pipeline, tamanho de código e energia.

Para depurar:

```
PC, registradores, flags, memória, SP e LR
```

Procure o primeiro ponto em que o estado deixa de corresponder ao esperado.

18. Datapath e unidade de controle

18.1 Distinção central

datapath → transporta, armazena e transforma dados
 controle → gera sinais que comandam o datapath

Blocos usuais:

PC
 memória de instruções
 banco de registradores
 ULA
 memória de dados
 MUXes
 extensor de sinal
 registradores de pipeline, quando houver

18.2 Caminho das instruções

Instrução	Ação
ADD	lê dois registradores, ULA soma, grava registrador
ADDI	lê registrador e imediato, ULA soma, grava registrador
LOAD	ULA calcula endereço, memória lê, grava registrador
STORE	ULA calcula endereço, memória escreve, não grava registrador
BEQ	ULA/comparador verifica igualdade e pode alterar PC

Sinais típicos:

RegWrite → habilita escrita no banco
 MemRead → leitura da memória de dados
 MemWrite → escrita da memória de dados
 ALUSrc → escolhe registrador ou imediato
 MemToReg → escolhe ULA ou memória para write-back
 Branch/Jump → participa da escolha do próximo PC
 ALUOp/ALUControl → seleciona a operação da ULA

Regras:

LOAD → MemRead=1 e RegWrite=1
 STORE → MemWrite=1 e RegWrite=0
 BEQ → compara, mas não grava a subtração

18.3 Monociclo, multiciclo e pipeline

monociclo → toda instrução cabe em um ciclo longo
 multiciclo → instrução usa várias etapas/ciclos
 pipeline → etapas de instruções diferentes se sobrepõem

A unidade de controle coordena. Ela não substitui a ULA, o banco de registradores nem a memória.

controle principal → decodifica opcode e gera sinais gerais
 controle da ULA → combina ALUOp e campo de função
 hardwired → tende a ser mais rápido e menos flexível
 microprogramado → usa memória de controle e tende a ser mais flexível

Em um processador multiciclo, o controle costuma ser modelado como uma FSM.

19. Pipeline

19.1 Cinco estágios clássicos

IF → busca da instrução
 ID → decodificação e leitura de registradores
 EX → ULA/endereço/comparação
 MEM → acesso à memória de dados
 WB → escrita no registrador

Pipeline melhora principalmente a **vazão**. Não garante menor latência para uma instrução individual.

19.2 Fórmulas

Para n instruções e k estágios, idealmente:

$$\text{ciclos} = k + n - 1$$

Com penalidades:

$$\begin{aligned} \text{ciclos reais} &= k + n - 1 \\ &+ \text{ciclos de stall} \\ &+ \text{ciclos perdidos por flush/misprediction} \end{aligned}$$

Clock:

$$T_{\text{clock}} \approx \text{atraso do estágio mais lento} + \text{overhead do registrador}$$

Métricas:

$CPI = \text{ciclos} / \text{instruções}$
 $IPC = \text{instruções} / \text{ciclos}$
 $\text{tempo de CPU} \approx \text{instruções executadas} \times CPI \times T_{\text{clock}}$

Depois do enchimento, um pipeline ideal pode concluir aproximadamente uma instrução por ciclo. Isso não vale durante fill/drain nem diante de hazards.

Relações ideais aproximadas:

$\text{latência de uma instrução} \approx k \times T_{\text{clock}}$
 $\text{throughput após enchimento} \approx 1/T_{\text{clock}}$
 $\text{speedup} = \text{tempo sem pipeline} / \text{tempo com pipeline}$

19.3 Latência versus throughput

latência → tempo de uma instrução/operação
 throughput → quantas terminam por unidade de tempo

Mais estágios podem permitir **período de clock menor e frequência maior**, mas aumentam registradores, complexidade, custo de branches e latência em ciclos.

20. Hazards, stall, forwarding e flush

20.1 Tipos

estrutural → duas operações querem o mesmo recurso
 de dados → uma instrução depende de outra
 de controle → próximo PC ainda é incerto

Dependência mais comum no pipeline simples:

RAW → Read After Write

WAR e WAW normalmente não aparecem no pipeline clássico in-order de cinco estágios com leitura/escrita em posições fixas, mas podem aparecer em arquiteturas mais avançadas.

20.2 Forwarding

Encaminha um resultado diretamente de um estágio posterior para a entrada que precisa dele, antes do write-back.

Exemplo ALU-ALU:

ADD R1, R2, R3
 SUB R4, R1, R5

Forwarding costuma resolver sem esperar o WB.

Caminhos que podem aparecer em diagramas:

```
EX/MEM → EX
MEM/WB → EX
```

Forwarding só ajuda quando o valor já está disponível. Para um **LOAD**, o registro EX/MEM ainda contém o endereço; o dado carregado surge após **MEM**.

20.3 Load-use

```
LOAD R1,[R2]
ADD R3,R1,R4
```

O dado do load só fica disponível após **MEM**, mas a instrução seguinte precisa dele em **EX**. No pipeline clássico, normalmente é necessário:

```
1 stall + forwarding
```

Inserir uma instrução independente entre elas pode esconder a bolha.

20.4 Stall, bubble e flush

```
stall → faz a instrução esperar
bubble → entrada inválida/controles neutralizados, com efeito de NOP
flush → invalida e descarta instruções já buscadas
```

Uma bubble interna não precisa ser uma instrução **NOP** escrita no programa.

Stall preserva a instrução correta para que ela continue depois. Flush elimina instruções que não devem mais executar, como as do caminho errado após uma predição incorreta.

Normalmente o programador **não escreve** “stall” ou “flush” no código. A lógica de hazards e controle do processador gera esses sinais. O compilador pode reorganizar instruções ou, em arquiteturas antigas/específicas, inserir NOPs.

20.5 O que cada solução trata

```
forwarding → dependência de dados disponível a tempo
stall → dado/recurso ainda não disponível
duplicação de recurso → hazard estrutural
predição → tenta antecipar hazard de controle
flush → corrige caminho especulado errado
```

Diminuir o clock pode ajudar temporização elétrica, mas não remove uma dependência lógica entre instruções.

21. Branch prediction

21.1 Conceitos

tomado → PC vai ao alvo
 não tomado → PC segue sequencialmente

Esperar a decisão evita caminho errado, mas paralisa o pipeline. Predizer permite continuar especulativamente.

21.2 Estática e dinâmica

estática → regra fixa, sem histórico
 dinâmica → usa comportamento observado

“Sempre tomado” é uma predição **estática**, mesmo que pareça uma estratégia sobre branches.

Preditor de 1 bit:

repete o último resultado

Históricos:

local → comportamento anterior daquele branch
 global → resultados recentes de vários branches
 aliasing → branches diferentes compartilham a mesma entrada

Preditor de 2 bits:

00 fortemente não tomado
 01 fracamente não tomado
 10 fracamente tomado
 11 fortemente tomado

00/01 → prevê não tomado
 10/11 → prevê tomado

O contador satura em 00 e 11. Partindo de um estado forte, são necessários dois resultados contrários consecutivos para virar a previsão; de um estado fraco, um basta.

21.3 Estruturas

BHT/PHT → ajudam a prever direção
 BTB → guarda/fornece alvo
 RAS → prevê endereço de retorno de função

Direção correta e alvo correto são duas perguntas diferentes. BTB não executa a condição do branch.

$\text{acurácia} = \text{predições corretas} / \text{branches previstos}$
 $\text{taxa de erro} = 1 - \text{acurácia}$

21.4 Custo

$\text{CPI} \approx \text{CPI}_{\text{ideal}}$
 $+ \text{frequência de branches} \times \text{taxa de erro} \times \text{penalidade}$

Predição errada causa flush das instruções jovens do caminho errado. O estado arquitetural correto deve ser preservado; o erro de predição não muda o resultado final do programa.

22. FPU em hardware

22.1 Função

FPU é a unidade que executa operações em ponto flutuante.

precisão simples → 32 bits
 precisão dupla → 64 bits

Suporte a múltiplas precisões é uma característica real de FPU. Cache e memória virtual pertencem a outras partes do sistema.

22.2 Fluxo de uma operação

desempacotar
 → classificar operandos
 → operar
 → normalizar
 → arredondar
 → empacotar

Soma/subtração:

alinhar expoentes
 somar/subtrair significandos
 normalizar
 arredondar

Multiplicação:

multiplicar significandos
 somar expoentes e corrigir bias

Divisão:

dividir significandos
subtrair expoentes e corrigir bias

Na soma, os expoentes são **alinhados**, não somados.

22.3 Casos e flags

Casos IEEE 754:

zero, subnormal, normal, infinito, NaN

Flags possíveis:

operação inválida
divisão por zero
overflow
underflow
inexato

Conversão numérica entre tipos não é a mesma coisa que reinterpretar os mesmos bits.

22.4 Desempenho

latência → tempo até o resultado de uma operação
throughput → frequência com que novos resultados podem sair

Operações independentes podem ocupar o pipeline da FPU em paralelo temporal. Operações dependentes precisam esperar o resultado anterior.

Dupla precisão oferece mais faixa/precisão, mas pode custar mais área, energia, banda e latência.

23. O que a prova anterior ensinou

23.1 Mapa dos conceitos cobrados

Questão anterior	Núcleo da resposta
Inglês 1	assunto principal, não detalhe verdadeiro
Inglês 8	<code>further</code> indica adição, próximo de <code>moreover</code>
11	cache associativa por conjunto
14	controle gera sinais para o datapath
15	“sempre tomado” é predição estática
16	imediato é constante na instrução
18	FPU pode suportar múltiplas precisões
19	antecipar dados é prefetch
20	branch altera o fluxo
21	subtrator realiza o decremento
22	hazards exigem detecção/tratamento no pipeline
23	temporários rápidos ficam em registradores
24	layout e caminhos afetam atraso
25	PROM tradicional é programada uma vez
26	mais bits fracionários melhoram resolução
27	FPGA oferece flexibilidade/reconfiguração
28	atrasos desiguais podem gerar glitch
29	CMOS favorece baixo consumo
30	ripple-carry liga <code>Cout</code> ao próximo <code>Cin</code>
51	contador síncrono para maior previsibilidade/desempenho
54	encoder decimal-BCD: 10 entradas, 4 saídas
55	mintermos são produtos das linhas de saída 1
56	FSM/circuito sequencial guarda estado
57	simplificação/identificação de XNOR
58	MUX seleciona uma entrada
59	binário-hexadecimal em grupos de 4 bits

Questão anterior	Núcleo da resposta
60	Karnaugh elimina a variável que muda

23.2 Como ler uma alternativa de outra camada

Na prova anterior, cache, FPU, registradores e memória virtual apareceram próximos como distratores. Use:

```

calcular com float      → FPU
guardar temporário rápido → registrador
reduzir latência de RAM → cache
buscar antes da demanda → prefetch
traduzir endereços      → memória virtual

```

23.3 Questões imperfeitas

O material anterior possui itens ambíguos ou incompletos:

- uma questão de inglês perdeu o destaque necessário;
- uma questão humorística permite interpretações próximas;
- a questão de contagem regressiva admite defender o oscilador como referência temporal, embora a banca tenha priorizado o subtrator;
- algumas descrições de assembly se sobrepõem.

Na prova real:

1. identifique a intenção mais provável;
2. escolha a alternativa mais direta e específica;
3. não deixe uma questão ambígua consumir o tempo de três questões claras;
4. só recorra a uma interpretação exótica se as opções comuns falharem.

24. Folha de fórmulas

24.1 Quantidade, capacidade e faixas

```

n bits                → 2^n combinações
tabela com n variáveis → 2^n linhas
decoder n→2^n
MUX com 2^n entradas  → n seletores
n bits de endereço    → 2^n posições
capacidade em bits     → 2^n × largura da palavra
capacidade em bytes    → capacidade em bits / 8
LUT de k entradas      → 2^k posições
N estados em binário   → ceil(log2 N) FFs
N estados one-hot      → N FFs
ring com n FFs         → n estados
Johnson com n FFs     → 2n estados

```

```

unsigned n bits          → 0 ... 2^n-1
signed C2 n bits         → -2^(n-1) ... 2^(n-1)-1
C=1 após subtração ARM   → sem borrow
C=0 após subtração ARM   → houve borrow

```

```

overflow em A-B signed:
A e B têm sinais diferentes
e o resultado tem sinal diferente de A

```

24.2 Booleana e blocos

```

(A+B)' = A'B'
(AB)'  = A'+B'
XOR    = A'B+AB'
XNOR   = AB+A'B'
MUX 2:1: Y=S'I0+SI1

```

```

Half-adder:
S=A XOR B
C=AB

Full-adder:
S=A XOR B XOR Cin
Cout=AB+A.Cin+B.Cin

```

24.3 Temporização

Com $\text{skew} = \text{clock_destino} - \text{clock_origem}$:

```

Tclock ≥ tCQmax+tcombmax+tsetup-skew
Fmax = 1/Tclock_min
Fmax(MHz)=1000/T(ns)

tCQmin+tcombmin ≥ thold+skew

```

```

skew positivo → ajuda setup, piora hold
skew negativo → piora setup, ajuda hold

```

Ripple-carry:

```

t_soma_pior = (n-1)tcarry + tsoma_final

```

24.4 Ponto fixo e IEEE 754

valor real = inteiro armazenado / 2^F
 resolução = 2^{-F}

IEEE simples:

valor normal = $(-1)^s \times 1.f \times 2^{(E-127)}$

24.5 Eletricidade e energia

$V=RI$

$P=VI$

$E=Pt$

$Q=CV$

$P_{din} \approx \alpha CV^2 f$

$P_{estática} \approx V \cdot I_{leak}$

$E_{cap} = 1/2 CV^2$

$E_{fonte \text{ no } 0 \rightarrow 1} = CV^2$

Atalhos de unidade:

$1 \text{ pF} \cdot V^2 = 1 \text{ pJ}$

$1 \text{ fF} \cdot V^2 = 1 \text{ fJ}$

24.6 Cache e desempenho

miss rate = $1 - \text{hit rate}$

conjuntos = capacidade / (bloco $\times$ vias)

offset = $\log_2(\text{bytes}/\text{bloco})$

índice = $\log_2(\text{conjuntos})$

tag = bits restantes

Se a questão fornecer tempos:

$AMAT = \text{hit time} + \text{miss rate} \times \text{miss penalty}$

24.7 Pipeline

ciclos ideais = $k+n-1$

ciclos reais = $k+n-1 + \text{ciclos de stall} + \text{ciclos perdidos em flush}$

$CPI = \text{ciclos}/\text{instruções}$

$IPC = \text{instruções}/\text{ciclos}$

tempo de CPU $\approx \text{instruções} \times CPI \times T_{clock}$

$$\text{CPI} \approx \text{CPI}_{\text{ideal}} + \text{frequência de branches} \times \text{taxa de erro} \times \text{penalidade}$$

24.8 Conversões

$$1 \text{ byte} = 8 \text{ bits}$$

$$1 \text{ word de 32 bits} = 4 \text{ bytes}$$

$$1 \text{ ms} = 10^{-3} \text{ s}$$

$$1 \mu\text{s} = 10^{-6} \text{ s}$$

$$1 \text{ ns} = 10^{-9} \text{ s}$$

$$1 \text{ ps} = 10^{-12} \text{ s}$$

$$1 \text{ kHz} = 10^3 \text{ Hz}$$

$$1 \text{ MHz} = 10^6 \text{ Hz}$$

$$1 \text{ GHz} = 10^9 \text{ Hz}$$

$$1 \text{ mW} = 10^{-3} \text{ W}$$

$$1 \mu\text{W} = 10^{-6} \text{ W}$$

$$1 \text{ pF} = 10^{-12} \text{ F}$$

$$1 \text{ fF} = 10^{-15} \text{ F}$$

Atalhos:

$$1000 \text{ ps} = 1 \text{ ns}$$

$$T(\text{ns}) = 1000 / f(\text{MHz})$$

$$f(\text{MHz}) = 1000 / T(\text{ns})$$

25. Mapa explícito das aulas 1-36

Use esta tabela para conferir cobertura. Se não conseguir explicar a frase de uma aula, volte à seção indicada deste arquivo ou à aula completa.

Aula	Lembrete indispensável
1	binário, complemento de 2, faixas, carry e overflow
2	BCD por dígito, Gray de um bit, ASCII, paridade e checksum
3	condição de cada porta; XOR é diferente, XNOR é igual
4	leis booleanas, absorção e distributivas
5	De Morgan; NAND-NAND para SOP e NOR-NOR para POS
6	mintermo vem dos 1; maxtermo vem dos 0
7	Karnaugh 2/3 variáveis: Gray, potências de 2 e bordas
8	Karnaugh 4 variáveis: cantos, wrap-around e don't care
9	especificação → tabela → minimização → circuito
10	MUX seleciona; decoder ativa; encoder codifica
11	Moore depende do estado; Mealy também da entrada
12	latch é sensível a nível; SR-NOR 11 é inválido
13	FF D copia na borda; registradores e tipos de deslocamento
14	síncrono compartilha clock; ripple acumula atraso
15	setup usa caminho máximo; hold, mínimo; atenção ao skew
16	com FF D, $D=Q+$; binário versus one-hot
17	ponto fixo usa escala; float usa sinal, expoente e fração
18	half/full-adder; ripple liga carries; carry $\neq$ overflow
19	$A-B=A+B'+1$; flags; LSL/LSR/ASR; ULA
20	NMOS liga com 1; PMOS com 0; $P=VI$, $E=Pt$
21	saída CMOS é o complemento da condição da pull-down
22	$\alpha CV^2 f$, energia do capacitor, atraso e glitches
23	registrador/SRAM/DRAM/ROM; capacidade por endereço e palavra
24	PROM/PLA/PAL; LUT, CPLD, FPGA, ASIC e bitstream
25	opcode, operandos, registradores, imediato, load/store
26	formatos e endereço efetivo: base, offset, índice e PC

Aula	Lembrete indispensável
27	controle gera sinais; datapath executa o fluxo de dados
28	cache: hit/miss, localidade, mapeamento, tag/índice/offset
29	prefetch: next-line/stream/stride; accuracy/coverage/timeliness
30	IF-ID-EX-MEM-WB; throughput, CPI e $k+n-1$
31	estrutural/dados/controle; forwarding, stall, bubble e flush
32	estática não usa histórico; 2 bits tem histerese; BTB dá alvo
33	FPU alinha/normaliza/arredonda; GRS, casos e flags IEEE
34	otimização preserva significado; memória, chamadas e depuração
35	CMP/branches/flags; <code>[]</code> , <code>#</code> , <code><<2</code> , SP, LR e endianness
36	main topic, conectivos, paráfrase, objetivo e evidência textual

26. Pegadinhas que merecem uma última olhada

Carry final não é overflow signed.
 C=1 em subtração ARM significa sem borrow.
 Estado atual Q não é próximo estado Q+.
 Com FF D, D recebe Q+.
 Latch não é flip-flop.
 One-hot gasta N FFs para N estados.
 Ring tem n estados; Johnson, 2n.
 Setup olha o caminho máximo; hold, o mínimo.
 Reduzir frequência não costuma corrigir hold.
 Pull-down conduz quando a saída deve ser zero.
 Ecap é $1/2CV^2$; a fonte fornece CV^2 no carregamento.
 Glitch elétrico não é hazard de pipeline.
 SRAM é volátil mesmo sem refresh.
 Cache não é memória virtual.
 Prefetch pode piorar o desempenho.
 Pipeline melhora vazão, não necessariamente latência.
 Forwarding não resolve todo load-use.
 Stall espera; flush descarta.
 Sempre tomado é predição estática.
 BTB fornece alvo, não decide a condição.
 CMP atualiza flags e não guarda a subtração.
 BNE consulta Z; não compara sozinho.
 [R1] é memória no endereço R1, não o próprio R1.
 <<2 multiplica por 4; não soma 2.
 BLT signed não é BLO unsigned.
 Endianness muda ordem de bytes, não de bits.
 FPU não é cache.
 Na soma float, alinham-se os expoentes.
 Moreover/further adicionam; in short resume.
 Detalhe verdadeiro não substitui o assunto principal.

27. Se eu lembrar apenas de 20 coisas

1. Leia **incorreta**, **principal** e **diretamente** antes do contexto.
2. Responda mentalmente antes de olhar as alternativas.
3. Identifique a camada: software, assembly, microarquitetura, circuito ou temporização.
4. Carry unsigned e overflow signed são diferentes.
5. Em ARM, **C=1** após subtração significa sem borrow.
6. **D=Q+** na síntese com flip-flop D.
7. Setup: caminho máximo; hold: caminho mínimo.
8. Skew positivo ajuda setup e piora hold, na convenção adotada.
9. Ripple-carry: o carry percorre estágio por estágio.
10. Pull-down descreve quando a saída é zero; negue para obter **y**.

11. $P_{din} \approx \alpha CV^2 f$; tensão aparece ao quadrado.
12. $E_{cap} = 1/2 CV^2$; a fonte entrega CV^2 no $0 \rightarrow 1$.
13. Cache guarda cópias; prefetch antecipa; memória virtual traduz.
14. Controle emite sinais; datapath movimenta e opera dados.
15. CMP atualiza flags; BEQ usa $Z=1$, BNE usa $Z=0$.
16. $[base + índice \ll 2]$ usa índice vezes quatro.
17. Pipeline melhora throughput; load-use costuma exigir um stall.
18. Stall espera; flush descarta caminho errado.
19. "Sempre tomado" é estático; preditor dinâmico usa histórico.
20. No inglês, classifique o conectivo e escolha a ideia, não a palavra repetida.

28. Checklist da terceira passada

Para cada questão ainda duvidosa:

- ☐ O comando pede correta ou incorreta?
- ☐ A alternativa responde ao verbo principal?
- ☐ Estou escolhendo um detalhe verdadeiro?
- ☐ Signed ou unsigned?
- ☐ Carry, borrow ou overflow?
- ☐ Bits, bytes, words ou posições?
- ☐ O valor é conteúdo, endereço ou imediato?
- ☐ O offset precisa ser escalado?
- ☐ Estado atual ou próximo estado?
- ☐ Caminho máximo ou mínimo?
- ☐ A unidade está em ns ou ps?
- ☐ A rede dada é pull-down ou a função de saída?
- ☐ É cache, prefetch, registrador ou memória virtual?
- ☐ É hazard elétrico ou de pipeline?
- ☐ É stall ou flush?
- ☐ Branch: direção, alvo ou condição?
- ☐ No inglês, há evidência textual direta?

29. Fechamento para o dia da prova

Seu resultado ao fazer a prova anterior rapidamente e com distrações não mede seu teto. Na prova presencial, o ganho esperado vem de:

- usar o tempo disponível;
- aplicar as três passadas;
- transformar o cenário em uma pergunta curta;
- conferir as unidades e a camada do conceito;
- não trocar uma resposta sem evidência.

Você não precisa acertar a questão do modo mais elegante. Precisa identificar o conceito pedido, eliminar os distratores e registrar a alternativa correta com calma.

Na última leitura, não tente aprender um capítulo novo. Reative estas relações, durma adequadamente e chegue com margem de tempo.